Atividade 2 - Modelos de Regressão e Ensembles (MO432/MC934)

Luís Antônio Almeida Lima Vieira
Instituto de Computação
Universidade Estadual de Campinas
l221045@dac.unicamp.br

I. Projeto Prático - Previsão de Preços de Imóveis e Interpretabilidade

A. Introdução

Neste projeto, foram testadas diferentes famílias de modelos de Aprendizado Supervisionado no contexto de regressão, de forma a comparar o desempenho e extrair informações a partir do melhor modelo obtido. Foram testados modelos lineares, baseados em Máquinas de vetores suporte e modelos de Ensemble. Como fonte de dados, foi utilizado o dataset de preços de casas na Califórnia [2], constituído pelos preços medianos de imóveis e outras informações, coletadas no censo dos distritos da Califórnia em 1990, sendo uma base de dados muito famosa no ensino de conceitos introdutórios de Aprendizado de Máquina.

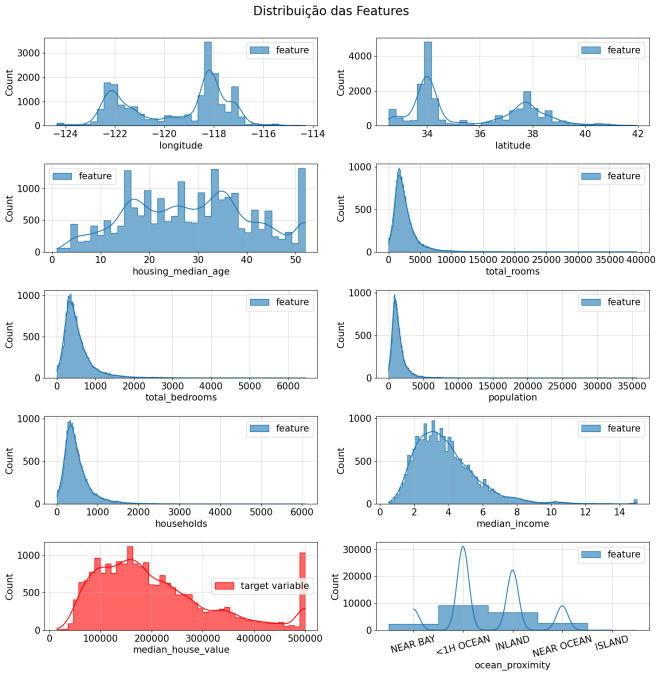

Fig. 1: Distribuição das variáveis do dataset de preços de casas na Califórnia.

Nestes dados, há informações coletadas que servem de features sobre as regiões das casas, como a latitude, longitude, idade mediana das casas na região (housing_median_age), total de cômodos das casas na região (total_rooms), total de quartos (total_bedrooms), população total que vive na região (population), o número de casas ocupadas na região (households), a renda média da região (median_income), e a proximidade com o oceano (ocean_proximity), sendo esta última uma variável categórica, indicando estados como próximo de uma baía, menos de 1 hora do oceano, no interior, próximo do oceano e em uma ilha. A variável alvo do destes dados é o preço mediano das casas em cada região (em dollars). A Figura 1 mostra as distribuições dos valores das variáveis do dataset.

B. Metodologia

Primeiramente, foi realizada uma análise exploratória dos dados. Foram extraídas e analisadas as distribuições das variáveis disponíveis (Fig. 1), permitindo identificar similaridades em algumas distribuições. Após isso, foi realizada uma análise de correlações entre as variáveis (Pearson Correlation), em que foi constatadas uma alta correlação entre as features total_rooms, total_bedrooms, population e households, além de uma alta correlação entre a variável alvo (preço médio) e a feature median_income.

Após esta análise, foi realizado um pré-processamento dos dados, que consistiu na verificação da presença de valores nulos (foram encontrados 207) e não numéricos (valores categóricos na coluna ocean_proximity). As linhas com valores nulos foram desconsideradas, e foi realizado um encoding (one-hot) dos valores categóricos, criando novas colunas binárias para essas categorias. Por fim, foi feita a separação de 20% dos dados em um conjunto de testes, para avaliar os modelos treinados visando analisar a capacidade de generalização em dados novos, seguida por um escalonamento (das variáveis não categóricas) nos conjuntos obtidos separadamente (para evitar vazamentos de dados), por meio de um processo de padronização (Standard Scaler). Para validação dos dados durante o treinamento, foi utilizado o método K-fold com 5 pastas obtidas do split de treino.

Feito isso, partindo da mesma estrutura de dados, foram testadas duas abordagens de modelagem para a regressão e previsão dos valores dos imóveis. Como forma de avaliar o desempenho dos modelos durante o treinamento, foram utilizadas como métricas de avaliação o Coeficiente de Determinação (R2 score), que mede o acompanhamento da variância dos

dados pelas predições do modelo, e o Erro Quadrático Médio, que mede a diferença entre os valores preditos e reais. Foi realizada a busca de hiperparâmetros ótimos durante o treinamento, ao avaliar as métricas de avaliação médias nos conjuntos de validação, e as mesmas métricas de avaliação foram usadas para medir o desempenho dos modelos treinados no conjunto de testes. Foi utilizada a busca de hiperparâmetros Bayesiana, por ser uma estratégia mais eficiente que a busca em grade, que testa todas as combinações de parâmetros.

1) Abordagem 1: Modelos Lineares e Support Vector Regression (SVR): Nessa abordagem, primeiramente foi treinado um modelo de Regressão Linear com regularização Ridge, servindo como baseline de desempenho. Para a definição do coeficiente de regularização α ótimo, foram testados diferentes valores em potência de 10 (de 0.0001 até 10000).

Após isso, foi treinado um modelo de Suport Vector Regressor (SVR), baseado em máquinas de vetores suporte. Foram testados valores do hiperparâmetro C que controla o grau de aceitação à violações de separabilidade da margem (variando de 0.01 a 1000 em potência de 10), o hiperparâmetro epsilon que representa a margem de tolerância ao erro de separação (variando de 0.01 a 1 uniformemente), além do tipo de função Kernel, testado como linear, polinomial e Gaussiano (RBF). O impacto de cada hiperparâmetro foi avaliado ao fixar os outros e variar apenas um, após obter a configuração ótima.

2) Abordagem 2: Modelos de Ensemble Baseados em Árvores: Nesta abordagem, foram testados modelos de Ensemble com árvores de decisão como modelos base sendo agrupadas para realizar as predições. Primeiro, foi treinado um modelo de Random Forest, que é um algoritmo de bagging, com aprendizado em paralelo baseado em bootstrap dos dados. Foi realizada uma busca de hiperparâmetros, avaliando o número de estimadores (número de árvores de decisão) (variando de 50 a 100 uniformemente), a profundidade máxima de cada árvore (variando de 3 a 30 uniformemente), número mínimo de amostras para ramificar as árvores (variando de 2 a 20 uniformemente), o número mínimo de amostras por folha das árvores (variando de 1 a 10 uniformemente) e o número máximo de features consideradas em cada nó (variando entre não definir, utilizar log na base 2 do número total de features e utilizar raíz do número de features.

Por último, foi testado o algoritmo XGBoost, do tipo Gradient Boosting, em que árvores de decisão são treinadas sequencialmente para reduzir os erros de árvores anteriores por meio do cálculo de gradiente. Foi realizada uma busca de hiperparâmetros, testando valores do número de árvores (de 50 a 500 uniformemente), profundidade máxima de cada árvore (de 3 a 15 uniformemente), taxa de aprendizado que define o passo do gradiente e a contribuição de cada árvore (variando de 0.01 até 0.3 uniformemente), subsample que define a porcentagem de dados de treino selecionada aleatoriamente por árvore (variando de 50% até 100%), a fração de features selecionadas para construir cada árvore (variando de 0.5 a 1 uniformemente), e o peso mínimo para um nó se ramificar (variando de 1 a 10 uniformemente).

Por fim, para o melhor modelo de Ensemble obtido, foi feita uma análise de importância das features e de interpretabilidade. Ao ordenar as features pela importância, ao mostrar como cada uma contribuiu para as decisões feitas pelas árvores, avaliando como cada variável contribuiu para a redução da função perda. Foram criados também gráficos de dependência parcial das 3 features mais importantes, para mostrar a relação dos valores assumidos por essa feature com o valor predito pelo modelo.

C. Resultados

TABLE I: Métricas no conjunto de validação (média das 5 pastas) para a abordagem 1 na melhor configuração.

Modelo	RMSE	R^2
Ridge	\$ 68718.95	0.64
SVR	\$ 69994.39	0.63

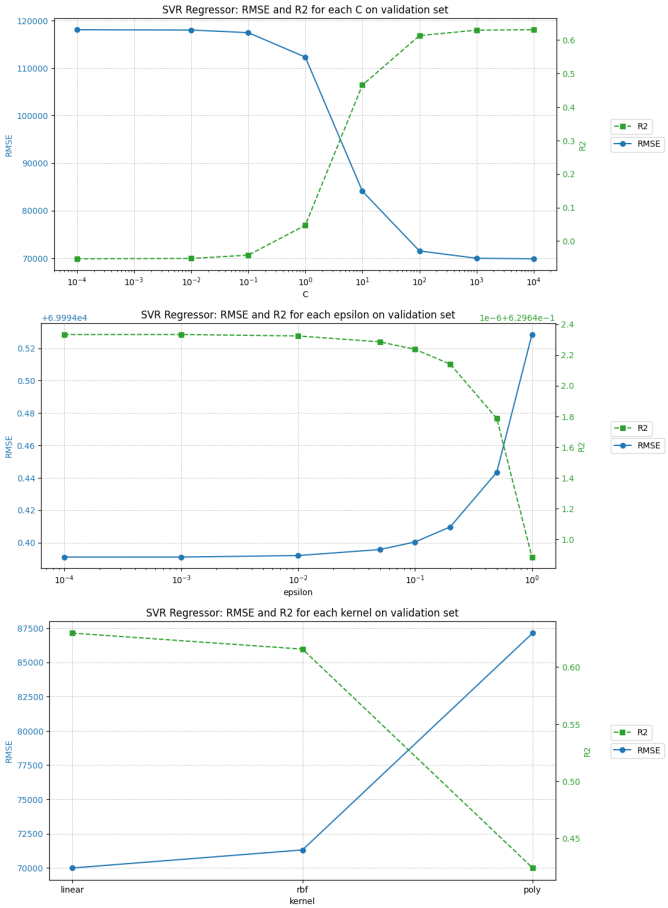

Fig. 2: Impacto da variação dos parâmetros do SVR nas métricas de validação.

TABLE II: Métricas no conjunto de validação para a abordagem 2 na melhor configuração de hiperparâmetros.

Modelo	RMSE	R^2
Random Forest Regressor	\$ 48820.07	0.82
XGBoost	\$ 45523.40	0.84

TABLE III: Métricas no conjunto de teste para a melhor configuração dos modelos testados.

Modelo	RMSE teste	R^2 teste
Ridge Regressor	\$ 69297.71	0.65
SVR	\$ 70874.86	0.63
Random Forest Regressor	\$ 48586.89	0.83
XGBoost	\$ 46176.13	0.84

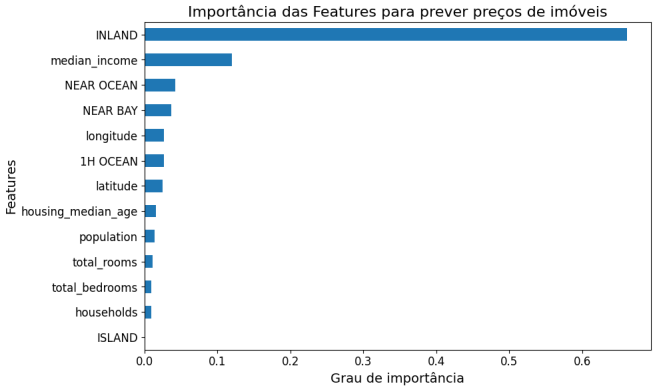

Fig. 3: Importância de todas as features do dataset para prever os valores dos imóveis na Califórnia.

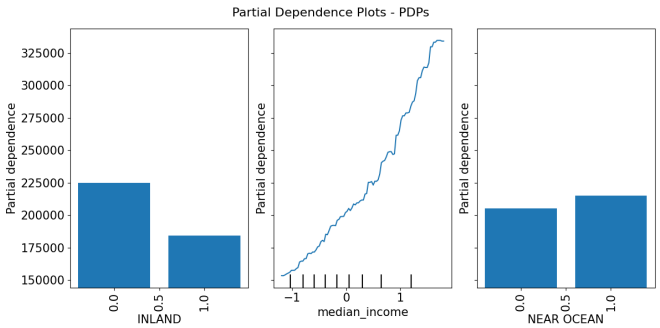

Fig. 4: Gráficos de dependência parcial (PDPs) para as 3 features mais importantes para prever o valor dos imóveis.

D. *Conclusão*

Ao comparar os 4 modelos testados, foi possível chegar a algumas conclusões com relação a eles. Com relação aos modelos da abordagem 1, percebeu-se uma semelhança entre as métricas (RMSE e R^2) de validação para as melhores configurações de hiperparâmetros. Com relação ao Ridge Regressor, as melhores métricas foram obtidas com baixos coeficiente de regularização α, porém o aumento deste coeficiente só iniciou a impactar o desempenho com valores acima de 100. Com relação ao SVR, como mostra a Figura 2, o desempenho de validação foi melhor para valores maiores de regularização C, não houve muito impacto do parâmetro epsilon ao manter as outras configurações ótimas, e o Kernel linear culminou em melhores métricas.

Para os modelos da abordagem 2 (Ensemble), foi observada uma melhoria nas métricas de validação, com relação à abordagem 1. As melhores métricas de validação para o Random Forest, foram obtidas com número de árvores maiores (410), com árvores mais profundas (23), e com valores próximos da metade dos hiperparâmetros testados para o Gradient Boosting, com 238 árvores, de profundidade 10, e com taxa de aprendizado pequena (0.05).

Ao avaliar os modelos no conjunto de testes (Tabela 3), foi possível perceber uma certa reprodução das métricas de validação, com destaque para a melhora de desempenho dos modelos da abordagem 2 com relação à abordagem 1. Os modelos de Ensemble tiveram desempenho similar, com os melhores resultados obtidos no Gradient Boosting, tanto no RMSE e R^2. Obteve-se com este modelo, um R^2 muito bom (próximo de 1) e um RMSE de \$ 46175,13, que é um erro razoável, considerando que a média dos valores dos imóveis é de \$ 206855,81, com valor máximo de \$ 500000,00 (erro de cerca de 9% do valor máximo e 22% do valor médio).

Com relação à análise de interpretabilidade, as features median_income, e as classes INLAND e NEAR_OCEAN de ocean_proximity, apresentam maior importância. O que faz sentido, considerando que foi observada na análise exploratória uma alta correlação entre median_income e o valor dos imóveis (median_house_value). Ao fazer o gráfico de dependência parcial para essas 3 features mais importantes (Figura 4), vemos uma relação decrescente da feature categórica INLAND, que significa que casas no interior tem valor de mercado menor, e uma relação crescente da feature categórica NEAR_OCEAN, que significa que casas próximas ao mar, possuem valores maiores. Com relação às dependências parciais da feature median_income (renda), vemos uma relação não linear, crescente, mas não monotônica, que se aproxima de uma reta crescente que mostra que pessoas de renda maior moram em imóveis com maior valor.

Por fim, com relação aos modelos testados, os modelos lineares tem como vantagens ser simples e rápidos de treinar, com menos risco de overfitting pela regularização, mas como desvantagens assumir relação linear entre as variáveis e a saída, não se adaptando à relações mais complexas. Já o modelo SVR tem como vantagem poder modelar não linearidades e ser robusto a outliers, mas como desvantagens a lentidão e complexidade computacional para treinamento. Já os Ensembles, capturam bem relações não lineares, são robustos à outliers, são mais eficientes que o SVR e permitem a análise de importância das features, mas tem como desvantagens a grande quantidade de hiperparâmetros, o que gera complexidade para busca do modelo ótimo, e são suscetíveis a overfitting se não forem bem regularizados. Possivelmente o modelo XGBoost teve a melhor performance por ter uma regularização eficiente (evitando overfitting), captar bem as relações não lineares como o median_income e a variável alvo, além de capturar a interação entre as variáveis.

II. Exercícios Teórico-Conceituais

A. ***Exercício 1: Máquinas de Vetores de Suporte e Kernels***

1) Problema de otimização do SVM de margem suave: Resposta

Para o problema do SVM de margem suave (soft-margin), dado um conjunto de treinamento $\{\mathbf{x}_i, y_i\}$ $i = 1, ..., N$, com $\mathbf{x}_i \in \mathbb{R}^n$ representando os dados e $y_i \in \{-1, +1\}$ a variável alvo, e uma variável de relaxação $\xi_i \geq 0$, que quantifica a tolerância a violações à hipótese de separabilidade dos dados, um hiperplano ótimo é obtido, buscando minimizar o nível exigido de tolerância de violações de forma a separar os dados com uma margem de separação. Com isso, o problema de otimização primal do SVM é formulado como:

$$\begin{aligned} &\min_{\mathbf{w}, b, \boldsymbol{\xi}} \quad \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N}\xi_i \\ &\text{sujeito a} \quad \begin{cases} y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, & i = 1, \ldots, N \\ \xi_i \geq 0, & i = 1, \ldots, N \end{cases} \end{aligned} \tag{1}$$

Sendo C um hiperparâmetro para regularização, $\mathbf{w}$ os parâmetros do modelo e b o termo de viés do hiperplano.

Para resolver o problema, são utilizados multiplicadores de Lagrange dadas as restrições, obtendo a formulação dual:

$$\begin{aligned} L(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu}) = \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N}\xi_i \\ -\sum_{i=1}^{N}\alpha_i\left[y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1 + \xi_i\right] - \sum_{i=1}^{N}\mu_i\xi_i \end{aligned} \tag{2}$$

Sendo α_i e μ_i os multiplicadores de Lagrange. As condições de KKT para o problema são mostradas abaixo, com $i = 1, ..., N$:

$$a_i \geq 0 \tag{3}$$

$$t_i y(\mathbf{x}_i) - 1 + \xi_i \geq 0 \tag{4}$$

$$a_i\left(t_i y(\mathbf{x}_i) - 1 + \xi_i\right) = 0 \tag{5}$$

$$\mu_i \geq 0 \tag{6}$$

$$\xi_i \geq 0 \tag{7}$$

$$\mu_i \xi_i = 0 \tag{8}$$

Derivando o Lagrangiano com relação as variáveis:

$$\frac{\partial L}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^{N}\alpha_i y_i \mathbf{x}_i = 0 \Rightarrow \mathbf{w} = \sum_{i=1}^{N}\alpha_i y_i \mathbf{x}_i \tag{9}$$

$$\frac{\partial L}{\partial b} = -\sum_{i=1}^{N}\alpha_i y_i = 0 \Rightarrow \sum_{i=1}^{N}\alpha_i y_i = 0 \tag{10}$$

$$\frac{\partial L}{\partial \xi_i} = C - \alpha_i - \mu_i = 0 \Rightarrow \alpha_i = C - \mu_i \tag{11}$$

Da equação 11, como $\mu_i \geq 0$ temos que $0 \leq \alpha_i \leq C$.

Ao substituir $\mathbf{w}$ (equação 9) na equação 2 e eliminando as variáveis b e ξ, é obtida a formulação dual da função objetivo (equação 12).

$$\begin{aligned} &L_{\text{dual}}(\boldsymbol{\alpha}) = \sum_{i=1}^{N}\alpha_i - \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N}\alpha_i\alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \\ &\text{sujeito a} \quad \begin{cases} 0 \leq \alpha_i \leq C, & i = 1, \ldots, N \\ \sum_{i=1}^{N}\alpha_i y_i = 0 \end{cases} \end{aligned} \tag{12}$$

2) Papel do hiperparãmetro C no SVM de margem suave: Resposta:

Como mostra a equação 1, no SVM de margem suave, enquanto o termo $\frac{1}{2}\|\mathbf{w}\|^2$ é responsável por maximizar a margem de separação entre os dados visando melhorar a generalização do modelo, o parâmetro C multiplica o termo de relaxação de violações de separabilidade na margem $\sum_{i=1}^{N}\xi_i$. Com isso, este hiperparâmetro funciona como um coeficiente de regularização para controlar a penalização para hiperplanos com violações de margem durante a otimização dos parâmetros do modelo. Quando C tende a 0, há pouca penalização para violações de margem, definidas por ξ_i, com isso, são obtidos hiperplanos ótimos com margens mais largas, com maior número de vetores suporte, pelo maior número de exemplos que violam a marge, caracterizando um risco de sob-ajuste do modelo (Underfitting). Por outro lado, quando C tende a infinito, há maior penalização a violações de margem, forçando a obter um hiperplano ótimo com margem estreita, com baixo número de vetores suporte, caracterizado um risco de Overfiting do modelo, ao especializar muito na separação específica dos dados de treinamento.

3) Truque do kernel: Resposta:

O truque do Kernel permite adaptar o problema para casos em que não há uma separabilidade linear entre os dados, necessitando de uma forma mais complexa para separar os dados. Este método consiste em realizar uma transformação (mapeamento) dos dados para um espaço de alta dimensão de forma que o SVM encontre um hiperplano que separe os dados linearmente neste espaço. Este mapeamento é realizado por uma função, porém não é necessário calcular este mapeamento para os dados diretamente, pois como na formulação dual do SVM (equação 12), os dados aparecem em produtos escalares entre amostras, é aplicada uma função Kernel que substitui esse produto escalar já em um espaço de alta dimensão, e nisso consiste o truque do Kernel.

O Kernel Polinomial cria um espaço de características (de maior dimensão) por meio de combinações polinomiais das entradas (funções polinomiais). Com isso, este tipo de Kernel permite criar fronteiras polinomiais, contendo curvas, como por exemplo uma parábola.

O Kernel Gaussiano (RBF), mapeia os dados para um espaço de dimensão infinita, aplicando funções de base radial (gaussianas), permitindo criar estruturas de separação complexas sem um padrão geométrico claro. Função Kernel RBF:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2\right), \quad \text{com } \gamma = \frac{1}{2\sigma^2} \tag{13}$$

B. *Exercício 2: Backpropagation e Diferenciação Automática*

1) Cálculo do gradiente da função de erro de soma dos quadrados: Resposta:

Dada uma rede neural, com uma camada de entrada $\mathbf{x} \in \mathbb{R}^D$, uma camada oculta de M neurônios com função de ativação sigmoide $z_j = \sigma(a_j)$ e uma camada de saída $y_k = a_k$, com a função de erro dada por:

$$E = \frac{1}{2}(y_k - t)^2$$

Para a derivação com relação a $w_{ji}^{(1)}$, é utilizada a regra da cadeia:

$$\frac{\partial E}{\partial w_{ji}^{(1)}} = \frac{\partial E}{\partial y_k} \cdot \frac{\partial y_k}{\partial a_k} \cdot \frac{\partial a_k}{\partial z_j} \cdot \frac{\partial z_j}{\partial a_j} \cdot \frac{\partial a_j}{\partial w_{ji}^{(1)}} \tag{14}$$

Para a camada de entrada, temos que

$$a_j = \sum_i w_{ji}^{(1)} x_i + b_j$$
$$z_j = \sigma(a_j) = \frac{1}{1 + e^{-a_j}}$$

Para a camada de saída, temos que

$$y_k = a_k = \sum_j w_{kj}^{(2)} z_j + b_k$$

Ao calcular a derivada de cada termo da regra da cadeia, temos que:

$$\frac{\partial E}{\partial y_k} = y_k - t$$
$$\frac{\partial y_k}{\partial a_k} = 1$$
$$\frac{\partial a_k}{\partial z_j} = w_{kj}^{(2)}$$
$$\frac{\partial z_j}{\partial a_j} = z_j(1 - z_j)$$
$$\frac{\partial a_j}{\partial w_{ji}^{(1)}} = x_i$$

Substituindo cada termo na equação 14:

$$\frac{\partial E}{\partial w_{ji}^{(1)}} = (y_k - t) \cdot w_{kj}^{(2)} \cdot z_j(1 - z_j) \cdot x_i \tag{15}$$

2) Diferença entre modo direto (foward-mode) e modo reverso (reverse-mode) da Diferenciação Automática: Resposta:

No modo direto é realizada a propagação das derivadas, pela regra da cadeia, partindo da camada de entrada para a saída da rede, enquanto no modo reverso (backpropagation), as derivadas são propagadas partindo da saída da para a entrada da rede, partindo da função custo (loss). O modo reverso é computacionalmente mais eficiente para treinar redes neurais com muitos parâmetros e uma única saída, pois iniciando da saída, calcula-se o gradiente com relação a todos os parâmetros simultaneamente, realizando apenas um caminho ao longo da rede. Por outro lado, no modo direto, seria necessário calcular a derivada de cada entrada até a saída, realizando diversos caminhos ao longo da rede, sendo necessário muito mais operações matemáticas.

3) Vanishing gradient: Resposta:

Durante o treinamento, é calculado o vetor gradiente da função custo com relação aos pesos, que define a direção e a intensidade com que os pesos devem ser atualizados no espaço para obter um valor ótimo da função custo. Em alguns casos o gradiente da função de perda pode se tornar muito pequeno, fazendo com que os pesos do modelo sejam atualizados muito lentamente. O uso de funções sigmoides para ativação torna contribui para ocorrer o desaparecimento de gradiente, pois a derivada desta função ($\sigma(x) = \frac{1}{1+e^{-x}}$), dada por $\sigma'(x) = \sigma(x)\,(1 - \sigma(x))$, tem valor máximo de 0.25, que é um valor baixo, e ao ser multiplicado durante o backpropagation, o resultado final decai e pode se tornar quase nulo. Já a função ReLU, é definida por:

$$\text{ReLU}(x) = \begin{cases} x & \text{se } x > 0 \\ 0, & \text{se } x \leq 0 \end{cases}$$

E sua derivada é dada por:

$$\text{ReLU}(x) = \begin{cases} 1 & \text{se } x > 0 \\ 0, & \text{se } x \leq 0 \end{cases}$$

Logo, para valores positivos, não há decaimento do gradiente durante as multiplicações, por possuir valor 1, o que evita a obtenção de valores de gradiente muito baixos, que deixam a atualização dos pesos muito lenta.

C. *Exercício 3: Métodos de Ensemble*

1) Bagging e Boosting: Resposta:

Nos métodos de Ensemble, como o Bagging e o Boosting, múltiplos modelos mais simples são combinados na forma de um comitê para formar em conjunto um modelo mais robusto. Esses modelos mais simples podem ser quaisquer de qualquer natureza, porém no Bagging e no Boosting é comum utilizar um conjunto de árvores de decisão.

O Bagging funciona por meio da técnica de bootstrap, em que são obtidos subconjuntos dos dados de treinamento, com uma amostragem por reposição, em que os elementos sorteados são devolvidos ao conjunto antes da próxima amostragem o que permite que um elemento pode ser escolhido mais de uma vez. Para cada subconjunto de dados obtidos, é treinado um modelo base, independentemente dos outros modelos obtidos, caracterizando um processo em paralelo. Após treinados, as saídas dos modelos base são combinados durante uma predição, por exemplo por meio de voto majoritário no caso de classificação e média no caso de regressão, obtendo a predição do ensemble. No Bagging, a variância do modelo final é reduzida ao agregar modelos base que são treinados com dados ligeiramente diferentes, suavizando a predição e reduzindo a probabilidade de overfitting. Já o viés tende a ser pouco impactado por esse método, sendo próximo do viés dos modelos base.

O Boosting utiliza todo o conjunto de dados para treinar modelos base, sendo que o próximo modelo base a ser treinado tem o objetivo de corrigir os erros cometidos pelos modelos anteriores, funcionando, assim, de forma sequencial, com os modelos base sendo dependentes entre si, atribuindo para um próximo modelo um peso maior para as amostras com piores predições dos modelos anteriores, ajustando-o para focar nessas amostras. Ao final, as predições de todos os modelos base são combinadas, por meio de, por exemplo uma média ponderada, dando peso maior para os melhores modelos. No Boosting, portanto, o viés é reduzido pela correção sequencial dos erros, porém a variância pode ser impactada, aumentando o risco de overfitting, em caso de modelos base complexos ou muitos modelos base.

2) Gradient Boosting: Resposta:

O Gradient Boosting, é uma técnica de Ensemble do tipo Boosting, em que modelos mais simples são treinados sequencialmente corrigindo os erros de predição dos modelos anteriores, obtendo um modelo forte com a combinação desses modelos mais simples. Os modelos bases desse Ensemble são árvores de decisão. Esse algoritmo funciona com uma função de perda, que depende do tipo de problema, para regressão, por exemplo, pode ser usado o erro quadrático médio. Um modelo base é ajustado a cada iteração para minimizar o valor da função de perda. Ele é chamado de ”Gradient” pois o processo de aprendizado ocorre pelo gradiente descendente da função custo, porém em vez de ajustar parâmetros, como ocorre nas redes neurais, são ajustadas funções para reduzir a perda. Essas funções podem ser árvores de decisão. Portanto, a cada iteração é calculado o gradiente da função de perda, indicando a direção de maior erro, e a próxima árvore de decisão é ajustada para aproximar este gradiente, dando um passo para reduzir o erro. Os pseudo-resíduos são o negativo do gradiente da função de perda com relação à predição do modelo atual. Com isso, a cada iteração a árvore é treinada para prever os pseudo-resíduos, mostrando em qual sentido a próxima árvore deve ser treinada para corrigir os erros das árvores anteriores. As previsões são multiplicadas por uma taxa de aprendizado e adicionadas com as predições anteriores, para formar um novo modelo. Com isso, é obtido um modelo final robusto, com a conjunção dos modelos menores, treinados sequencialmente a partir dos pseudo-resíduos, para ajustar modelos menores.

3) Partial Dependence Plots: Resposta:

Gráficos de Dependência Parcial (Partial Dependence Plots - PDP), mostram como uma feature do problema afeta a predição feita pelo modelo, sendo muito úteis para a interpretabilidade do modelo ao mostrar o impacto de determinada variável na predição, o que facilita o entendimento das decisões realizadas pelo algoritmo. Para uma determinada feature X_s, o PDP é calculado fixando a feature em um valor específico e substituindo este valor em todo o conjunto de dados para esta variável. Após esta modificação nos dados, são realizadas predições a partir dessas amostras modificadas e calculada a média de todos os valores preditos. Este processo é repetido fixando outros valores de X_s. Com isso, é feito um gráfico contendo no eixo x todos os valores testados da feature X_s, e no eixo y a média de resultados preditos pelo modelo para cada um desses testes ao fixar a feature para todo o conjunto de dados.

Por meio deste gráfico, é possível tirar algumas conclusões sobre a importância de X_s. Se for plotada uma linha ascendente, significa que o valor predito aumenta conforme o valor da feature aumenta, ou seja, o modelo entende que essa feature é essencial para o aumento da variável alvo. Por outro lado, se for uma linha decrescente, significa que o valor predito diminui com o aumento do valor da feature. Valores constantes do eixo y (média do valor predito) indicam que esta feature não tem efeito nas predições. O formato da curva também mostra o tipo de relação desta variável com o resultado do modelo, podendo ser linear ou não linear.

A principal limitação dos PDPs ocorrem quando há muita interação entre a feature X_s a ser plotada e as outras variáveis dos dados. Neste caso, ao fixar o valor de X_s pode ocorrer combinações não factíveis na realidade, gerando previsões para esses casos que não poderiam aparecer na realidade dos dados, o que pode levar a interpretações incorretas.

REFERENCES

[1] Bishop, Christopher M., and Nasser M. Nasrabadi. Pattern recognition and machine learning. Vol. 4. No. 4. New York: springer, 2006.

[2] Aurélien Géron's. Hands-On Machine learning with Scikit-Learn and TensorFlow. California Housing Prices Dataset. Kaggle, 2017. Disponível em https://www.kaggle.com/datasets/camnugent/california-housing-prices